CONSULTÓRIO PRIVADO

Guia de Arquitetura e Desenvolvimento

Documento de referência para programadores | Versão 2.0 | Outubro de 2026

Base documental: código-fonte da solução Consultorio.zip, quatro projetos .NET 10, nove entidades, oito controladores MVC, oito ViewModels, duas migrations e ficheiros de configuração. O conteúdo assinala o que foi observado no código e distingue recomendações futuras.

Índice

1. Visão geral

2. Arquitetura da solução

3. Estrutura do código

4. Modelo de dados

5. Fluxos de execução

6. Regras de negócio

7. Entity Framework Core

8. Tratamento de erros

9. Extensão da aplicação

10. Manutenção e evolução

Anexo A — Inventário de classes

Anexo B — Lista de verificação

1. Visão geral

Aplicação demonstrativa de gestão de consultório médico. Implementa registo e manutenção de utentes, médicos, especialidades, consultas, atos clínicos, tipos de atos, pagamentos, horários e indisponibilidades médicas, além de dashboard e agenda semanal. Os dados de demonstração são fictícios.

Tecnologia	Utilização comprovada
C# / .NET 10	Projetos com TargetFramework net10.0
ASP.NET Core MVC	Controllers, Views e rotas convencionais
Entity Framework Core 10.0.12	Persistência e consultas SQL Server
SQL Server	Provider UseSqlServer; base ConsultorioDemo
Razor / Bootstrap	Interface Web; ficheiros em Views e wwwroot
Git	Versionamento da solução

Âmbito: este documento descreve a implementação analisada, não uma arquitetura idealizada. Não foram identificados serviços próprios na camada Application; os controladores Web usam diretamente o contexto EF Core.

2. Arquitetura da solução

A solução está fisicamente organizada em quatro projetos, mas as dependências são verificadas a partir dos ficheiros .csproj:



Projeto	Responsabilidade real	Dependências
Domain	Entidades de negócio e propriedades	Sem referências a outros projetos da solução
Application	Projeto de organização aplicacional; sem serviços próprios identificados	Domain
Infrastructure	ConsultorioDbContext, mapeamentos e migrations	Domain, EF Core SQL Server
Web	Controllers MVC, ViewModels, Views, configuração e apresentação	Application, Infrastructure

Consequência arquitetural: o Web conhece diretamente Infrastructure. As regras de validação de consultas encontram-se em ConsultasController, pelo que a separação de responsabilidades é parcial. Isto é uma observação do estado atual, não um erro de compilação.

3. Estrutura do código

```
Consultorio/
  Consultorio.slnx
  Consultorio.Domain/Entities/
```

```
Consultorio.Application/
Consultorio.Infrastructure/Data/ConsultorioDbContext.cs
Consultorio.Infrastructure/Migrations/
Consultorio.Web/Controllers/
Consultorio.Web/Models/
Consultorio.Web/Views/
Consultorio.Web/wwwroot/
Consultorio.Web/Program.cs
```

3.1 Controladores e ações

Controlador	Ações públicas identificadas
AtosClinicosController.cs	Index, Details, Create, Edit, Anular
ConsultasController.cs	Index, Details, Create, Edit, AlterarEstado, Agenda
DashboardController.cs	Index
EspecialidadesController.cs	Index, Details
MedicosController.cs	Index, Details, Create, Edit, AlterarEstado, Horarios, CriarHorario, RemoverHorario, Indisponibilidades, CriarIndisponibilidade, RemoverIndisponibilidade
PagamentosController.cs	Index, Details, Create, Edit
TiposAtosController.cs	Index, Details, Create, Edit, AlternarEstado
UtentesController.cs	Index, Details, Create, Edit, AlterarEstado

3.2 Modelos de apresentação

Os seguintes ViewModels estão em Consultorio.Web/Models: AgendaSemanalViewModel.cs, AtoClinicoFormViewModel.cs, ConsultaFormViewModel.cs, CriarUtenteViewModel.cs, HorarioMedicoFormViewModel.cs, IndisponibilidadeMedicoFormViewModel.cs, MedicoFormViewModel.cs, PagamentoFormViewModel.cs, TipoAtoFormViewModel.cs. A sua finalidade é transportar dados entre os formulários e os controladores, incluindo os campos de validação do agendamento.

3.3 Configuração do arranque

```
using Consultorio.Infrastructure.Data;
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<ConsultorioDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("Consultorio")));
var app = builder.Build();
if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error"); app.UseHsts(); }
app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting();
app.MapControllerRoute(name:"default",pattern: "{controller=Dashboard}/{action=Index}/{id?}");
app.Run();
```

O Program.cs regista MVC e ConsultorioDbContext por injeção de dependências; lê ConnectionStrings:Consultorio; usa HTTPS, ficheiros estáticos e rota convencional Dashboard/Index. Não invoca Database.Migrate() nem EnsureCreated().

4. Modelo de dados

As nove classes de entidade estão em Domain/Entities. A estrutura SQL exportada contém ainda PerfisDemo, que não surge como DbSet no contexto analisado.

Entidade	Propriedades declaradas no código
AtoClinico	AtoId: int, ConsultaId: int, Consulta: Consulta?, TipoAtoId: int, TipoAto: TipoAto?, DataAto: DateTime, Descricao: string?, Valor: decimal, Anulado: bool
Consulta	ConsultaId: int, UtenteId: int, Utente: Utente?, MedicoId: int, Medico: Medico?, DataHora: DateTime, DuracaoMinutos: int, Estado: string, Observacoes: string?
Especialidade	EspecialidadeId: int, Designacao: string
HorarioMedico	HorarioMedicoId: int, MedicoId: int, Medico: Medico?, DiaSemana: DayOfWeek, HoraInicio: TimeSpan, HoraFim: TimeSpan, Ativo: bool
IndisponibilidadeMedico	IndisponibilidadeMedicoId: int, MedicoId: int, Medico: Medico?, DataHoraInicio: DateTime, DataHoraFim: DateTime, Motivo: string?
Medico	MedicoId: int, Nome: string, CedulaFicticia: string, EspecialidadeId: int, Especialidade: Especialidade?, Ativo: bool
Pagamento	PagamentoId: int, ConsultaId: int, Consulta: Consulta?, Valor: decimal, DataPagamento: DateTime, Metodo: string, ReferenciaFicticia: string
TipoAto	TipoAtoId: int, Codigo: string, Designacao: string, PrecoBase: decimal, Ativo: bool
Utente	UtenteId: int, Codigo: string, Nome: string, DataNascimento: DateTime, Telefone: string?, Email: string?, Localidade: string?, Ativo: bool

4.1 DbContext e mapeamento

ConsultorioDbContext expõe DbSet para Utentes, Especialidades, Medicos, TiposAtos, Consultas, AtosClinicos, Pagamentos, HorariosMedicos e IndisponibilidadesMedicos. Em OnModelCreating são configurados nomes de tabelas, chaves primárias, índices únicos, precisão decimal e tipos date/datetime2(0).

Índices únicos explicitamente configurados: Utente.Codigo, Medico.CedulaFicticia, TipoAto.Codigo e Pagamento.ReferenciaFicticia. Os horários e indisponibilidades têm índices compostos por médico e referência temporal. As duas relações explícitas com Medico nas tabelas de disponibilidade usam DeleteBehavior.Restrict.

4.2 Relações conceptuais

Utente (1) — (N) Consulta (N) — (1) Medico
Medico (1) — (N) HorarioMedico
Medico (1) — (N) IndisponibilidadeMedico
Consulta (1) — (N) AtoClinico
Consulta (1) — (N) Pagamento [confirmar cardinalidade no SQL]

O diagrama acima é conceptual: a cardinalidade exata e restrições de integridade devem ser lidas em conjunto com o script SQL de estrutura; não substitui um diagrama ER gerado da base.

5. Fluxos de execução

5.1 Criação de consulta

```
Browser / Views/Consultas/Create.cshtml
  │ POST /Consultas/Create + antiforgery token
  ▼
ConsultasController.Create(ConsultaFormViewModel)
  │ ValidarAgendamentoAsync(model)
  ├─► Utentes / Medicos: existência e estado ativo
  ├─► HorariosMedicos: horário semanal
  ├─► IndisponibilidadesMedicos: interseção temporal
  └─► Consultas: sobreposição de consultas agendadas
  │ ModelState válido?
  ├─ NÃO → repor listas e devolver View(model)
  └─ SIM → db.Consultas.Add → SaveChangesAsync → Redirect Details
```

No GET Create, o controlador carrega listas de utentes e médicos e devolve ConsultaFormViewModel. No POST Create, chama a validação, devolve a vista com erros se necessário e, caso contrário, grava uma entidade Consulta com Estado="Agendada". Usa TempData para mensagem de sucesso.

5.2 Edição e mudança de estado

Edit apenas permite editar consultas em estado Agendada. No POST é verificada a correspondência entre id da rota e ConsultaId, carregada a entidade, executada a validação e gravadas as alterações. AlterarEstado aceita apenas Realizada ou Cancelada e só altera consultas previamente Agendadas.

5.3 Agenda semanal

Agenda(DateTime? semana, int? medicoId) calcula a segunda-feira da semana de referência, define o limite exclusivo sete dias depois, filtra por médico quando solicitado, ordena as consultas e devolve AgendaSemanalViewModel.

6. Regras de negócio

6.1 Regras de agendamento

Regra	Implementação verificada
Utente e médico ativos	AnyAsync sobre Utentes e Medicos com Ativo=true
Data futura	data < DateTime.Now gera erro
Duração	Valores aceites: 15, 30, 45 ou 60 minutos
Horário semanal	Se existem horários ativos, consulta deve caber integralmente num intervalo do mesmo dia
Indisponibilidade	Conflito se inicioIndisponibilidade < fimConsulta e inicioConsulta < fimIndisponibilidade

Sobreposição	Consulta agendada do mesmo médico com início anterior ao fim e fim posterior ao início; exclui a consulta em edição

6.2 Excertos reais comentados

```
var fim = data.AddMinutes(model.DuracaoMinutos);

bool indisponivel = await db.IndisponibilidadesMedicos
   .AsNoTracking()
   .AnyAsync(i => i.MedicoId == model.MedicoId &&
            i.DataHoraInicio < fim && data < i.DataHoraFim);
```

A expressão utiliza intervalos semiabertos [início, fim): períodos contíguos, em que o fim de um coincide com o início do seguinte, não são considerados sobrepostos.

```
var inicioPesquisa = data.AddMinutes(-60);
// Procura consultas agendadas que podem intercetar o intervalo.
// A duração máxima atualmente admitida é de 60 minutos.
```

Ponto de manutenção: a janela de pesquisa retrospetiva fixa em 60 minutos está associada à duração máxima atual. Se futuramente forem permitidas consultas mais longas, a pesquisa terá de ser revista para não omitir conflitos.

Ponto de manutenção: a validação é efetuada na aplicação antes de SaveChangesAsync. Em pedidos concorrentes, duas operações podem validar simultaneamente o mesmo intervalo. A proteção contra concorrência deve ser estudada antes de uso real.

6.3 Comportamento quando não existem horários

A verificação do horário semanal só é aplicada quando existem registos ativos para o médico (horarios.Count > 0). Assim, um médico sem horários configurados não é bloqueado por essa regra. Deve decidir-se se este comportamento é intencional ou se é preferível exigir horários previamente definidos.

7. Entity Framework Core

7.1 Registo do contexto

```
builder.Services.AddDbContext<ConsultorioDbContext>(
   options => options.UseSqlServer(
      builder.Configuration.GetConnectionString("Consultorio")));
```

O contexto é resolvido pelo sistema de injeção de dependências do ASP.NET Core e recebido no construtor primário dos controladores. Consultas de leitura usam frequentemente AsNoTracking(); Include carrega referências como Utente e Medico; ToListAsync/FirstOrDefaultAsync materializam resultados; SaveChangesAsync persiste alterações.

7.2 Migrations e instalação

• 20261012163641_AdicionarDuracaoConsulta.cs — introdução da duração de consulta.

• 20261013095038_AdicionarDisponibilidadeMedicos.cs — estrutura de indisponibilidades médicas.

A base inicial foi criada por script SQL no SSMS; as migrations foram introduzidas posteriormente. O script ConsultorioDemo_Estrutura.sql representa o esquema final. Não se deve executar Update-

Database indiscriminadamente sobre uma base criada a partir desse esquema final, pois a tabela __EFMigrationsHistory não está incluída no script e o EF Core pode tentar repetir alterações.

Procedimento seguro para novo ambiente: criar uma base vazia, executar o script do esquema final, configurar a connection string e iniciar a aplicação sem aplicar migrations. Para futura evolução por migrations, reconciliar previamente o histórico EF Core com a estrutura instalada e validar numa cópia de teste.

8. Tratamento de erros

Mecanismo	Exemplo no código
ModelState.AddModelError	Mensagens de validação em DataHora, MedicoId, UtenteId e DuracaoMinutos
ModelState.IsValid	Bloqueia gravação e devolve formulário com erros
NotFound()	Registo inexistente em Details/Edit
BadRequest()	ID incoerente ou transição de estado inválida
TempData	MensagemSucesso e MensagemErro após redirecionamento
ValidateAntiForgeryToken	Proteção de formulários POST
UseExceptionHandler	Middleware configurado para ambientes não Development

Observação: Program.cs aponta para /Home/Error em ambientes não Development; não foi identificado HomeController no inventário de controladores do ZIP. Antes de publicação/produção, verificar se existe endpoint de erro efetivo e configurar uma página de erro funcional.

Não foi identificada uma camada centralizada própria de logging/gestão de exceções no código analisado. Para uma evolução, recomenda-se ILogger, mensagens de erro não sensíveis, tratamento de DbUpdateException e registo de eventos relevantes.

9. Extensão da aplicação

9.1 Exemplo: acrescentar nova entidade

1. Definir a entidade em Consultorio.Domain/Entities, com chave, propriedades e regras de domínio.

2. Acrescentar DbSet e mapeamento em ConsultorioDbContext; definir índices, precisão, chaves e relações.

3. Preparar a alteração do esquema de dados de forma compatível com o histórico híbrido SQL/migrations.

4. Criar ViewModel em Consultorio.Web/Models para os campos do formulário e respetivas validações.

5. Implementar controlador e ações GET/POST, usando ModelState, antiforgery e persistência assíncrona.

6. Criar Views Razor, navegação e mensagens de resultado.

7. Adicionar testes de criação, edição, integridade e cenários inválidos.

8. Atualizar README, guia de utilização e este guia técnico.

9.2 Exemplo de refatorização recomendada

```
public interface IAgendamentoService
{
    Task<ResultadoValidacao> ValidarAsync(
        ConsultaFormViewModel pedido, int? excluirId = null);
}
```

Exemplo conceptual — NÃO EXISTE no código atual. Numa evolução, mover a validação de ConsultasController para um serviço aplicacional e manter o controlador responsável por HTTP, ModelState e navegação. Evitar fazer depender Application de ViewModels da Web: definir um DTO/contrato próprio da camada Application.

10. Manutenção e evolução

Prioridade	Ação proposta	Motivo
Alta	Testes automatizados de sobreposição, horários e indisponibilidades	Evitar regressões em regras críticas
Alta	Rever concorrência no agendamento	Reduzir risco de dupla marcação
Alta	Resolver endpoint /Home/Error e tratamento de exceções	Melhorar diagnóstico e segurança
Média	Extrair regras de agendamento para Application	Melhor separação e testabilidade
Média	Reconciliar histórico SQL inicial / migrations	Permitir evolução controlada do esquema
Média	Validar comportamento sem horário configurado	Tornar a regra explícita
Média	Introduzir logs estruturados	Facilitar suporte e rastreabilidade
Futura	Autenticação, autorização e auditoria	Requisitos antes de uso com dados reais

As sugestões desta secção são melhorias futuras, não funcionalidades presentes no projeto. A POC não deve ser interpretada como aplicação pronta para exploração clínica com dados pessoais reais.

Anexo A — Inventário de ficheiros relevantes

Entidades: AtoClinico.cs, Consulta.cs, Especialidade.cs, HorarioMedico.cs, IndisponibilidadeMedico.cs, Medico.cs, Pagamento.cs, TipoAto.cs, Utente.cs.

Controladores: AtosClinicosController.cs, ConsultasController.cs, DashboardController.cs, EspecialidadesController.cs, MedicosController.cs, PagamentosController.cs, TiposAtosController.cs, UtentesController.cs.

Modelos: AgendaSemanalViewModel.cs, AtoClinicoFormViewModel.cs, ConsultaFormViewModel.cs, CriarUtenteViewModel.cs, HorarioMedicoFormViewModel.cs, IndisponibilidadeMedicoFormViewModel.cs, MedicoFormViewModel.cs, PagamentoFormViewModel.cs, TipoAtoFormViewModel.cs.

Persistência: ConsultorioDbContext.cs, migrations AdicionarDuracaoConsulta e AdicionarDisponibilidadeMedicos, ConsultorioDbContextModelSnapshot.cs.

Anexo B — Lista de verificação para programadores

☐ Abrir Consultorio.slnx e confirmar compilação dos quatro projetos.

☐ Configurar ConnectionStrings:Consultorio sem publicar credenciais.

☐ Garantir que a estrutura SQL corresponde ao modelo EF Core.

☐ Testar operações CRUD, filtros e paginação.

☐ Testar criação, edição e alteração de estado de consultas.

☐ Testar durações, horários, indisponibilidades e sobreposições.

☐ Confirmar página de erro em ambiente não Development.

☐ Verificar que dados e documentação públicos não expõem informação sensível.

Nota metodológica: inventário, dependências, regras e excertos foram retirados do ZIP da solução fornecido. As propostas de refatorização, testes e endurecimento operacional são recomendações técnicas, explicitamente separadas da implementação existente.